UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Observabilidade e Monitoramento de performance em aplicações de Machine Learning

Reyne Jasson Marcelino de Brito

Orientador: Prof. Dr. Carlos Manuel Dias Viegas

Natal - RN, 21 de janeiro de 2025

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Observabilidade e Monitoramento de performance em aplicações de Machine Learning

Reyne Jasson Marcelino de Brito

Orientador: Prof. Dr. Carlos Manuel Dias Viegas

Trabalho de Conclusão de Curso de Graduação na modalidade Monografia, submetido como parte dos requisitos necessários para conclusão do curso de Engenharia de Computação pela Universidade Federal do Rio Grande do Norte (UFRN/CT).

Natal, RN, 21 de janeiro de 2025

This work is licensed under a Creative Commons Attribution 4.0 International License

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Brito, Reyne Jasson Marcelino de.
Observalidade e Monitoramento de performance em aplicações de Machine Learning / Reyne Jasson Marcelino de Brito. - 2025.
40f.: il.

Monografia (Graduação) - Universidade Federal do Rio Grande do Norte, Centro de Tecnologia, Engenharia de Computação, Natal, 2025.
Orientação: Prof. Dr. Carlos Manuel Dias Viegas.

1. ElasticAPM - Monografia. 2. Machine Learning - Monografia. 3. monitoramento - Monografia. I. Viegas, Carlos Manuel Dias. II. Título.

RN/UF/BCZM CDU 004

Elaborado por Raimundo Muniz de Oliveira - CRB-15/429

Observalidade e Monitoramento de performance em aplicações de Machine Learning

Reyne Jasson Marcelino de Brito

Monografia aprovada em 20 de janeiro de 2025, pela banca examinadora composta pelos seguintes membros:

Prof. Dr. Carlos Manuel Dias Viegas (orientador) . . . . . DCA/UFRN

Prof. Dr. Eduardo de Lucena Falcão . . . . . DCA/UFRN

Prof. Dr. Ivanovitch Medeiros Dantas da Silva . . . . . DCA/UFRN

Às pessoas que não abandonam

Agradecimentos

À minha família, que foi a espinha dorsal de minha trajetória acadêmica, dedico minha mais sincera gratidão.

Ao meu orientador , professor Prof. Dr. Carlos Manuel Dias Viegas, sou grato pela orientação e pela paciência.

Agradeço aos colegas pela presença e apoio inestimáveis, que transformaram o ambiente acadêmico em um espaço de colaboração e crescimento mútuo.

À UFRN, expresso minha profunda gratidão. Em meio a desafios consideráveis, a dedicação de todos os seus membros — docentes, funcionários e colegas — à promoção do saber e ao apoio mútuo foi uma fonte inesgotável de inspiração e fortalecimento. A perseverança institucional em cultivar um ambiente de aprendizado e crescimento intelectual, mesmo sob pressões adversas, é um testemunho do compromisso inabalável com a educação e a pesquisa. Agradeço a todos os que, com sua competência e generosidade, contribuíram para a construção dos conhecimentos que sustentam esta dissertação.

Agradeço de maneira especial aos meus colegas de trabalho, cuja colaboração e expertise foram fundamentais ao longo desta jornada. A convivência e os conhecimentos compartilhados por eles não apenas enriqueceram este estudo, mas também moldaram minha compreensão e habilidades na área. Sem o apoio e as valiosas lições de cada um deles, a realização desta dissertação teria sido significativamente mais desafiadora. A gratidão por suas contribuições é imensa e reconheço aqui o papel essencial que desempenharam para o sucesso deste trabalho.

Resumo

Este trabalho apresenta a concepção e implementação de uma API REST conteinerizada voltada para servir um modelo de Machine Learning, e a integração dessa API com práticas de observabilidade e monitoramento utilizando o ElasticAPM e o Elasticsearch. O serviço de inferência é desenvolvido utilizando o framework FastAPI, escolhido por sua eficiência, facilidade de uso e versatilidade.

A instrumentação da API com o ElasticAPM visa coletar métricas detalhadas de performance, incluindo tempos de resposta, distribuição de spans e erros, permitindo uma análise aprofundada do comportamento da aplicação em tempo real. Esses dados de monitoramento são armazenados no Elasticsearch, que oferece uma solução escalável e eficiente para indexação e busca de grandes volumes de dados, facilitando a análise e a visualização através de ferramenta Kibana e o gerenciamento em ambientes voltados a microsserviços.

Palavras-chave: Machine Learning, API REST, FastAPI, ElasticAPM, Elasticsearch, MLops, observabilidade, monitoramento, conteinerização, Docker.

Abstract

This work presents the design and implementation of a containerized REST API designed to serve a Machine Learning model, along with the integration of this API with observability and monitoring practices using ElasticAPM and Elasticsearch. The inference service is developed using the FastAPI framework, chosen for its efficiency, ease of use, and versatility.

The instrumentation of the API with ElasticAPM aims to collect detailed performance metrics, including response times, span distribution, and errors, enabling in-depth analysis of application behavior in real-time. This monitoring data is stored in Elasticsearch, which offers a scalable and efficient solution for indexing and searching large volumes of data, facilitating analysis and visualization through the Kibana tool and management in microservice-oriented environments.

Keywords: Machine Learning, API REST, FastAPI, ElasticAPM, Elasticsearch, MLops, observability, monitoring, containers, Docker.

Sumário

Sumário i

Lista de Figuras iii

1 Introdução 1
- 1.1 Objetivos e motivação . . . 1
- 1.2 Estrutura do trabalho . . . 2

2 Fundamentação Teórica 3
- 2.1 Contêineres Docker . . . 3
- 2.2 Elasticsearch, Kibana e APM server . . . 3
- 2.3 FastAPI . . . 4

3 Desenvolvimento 7
- 3.1 Desenvolvimento da API REST . . . 7
- 3.2 Integração com ElasticAPM . . . 9
- 3.3 Dockerização dos Serviços e instrumentação de código . . . 12

4 Experimentos e Resultados 15
- 4.1 Configuração Experimental . . . 15
- 4.2 Visualização no Kibana . . . 16
- 4.3 *Insights* para Otimização . . . 17

5 Conclusão 19

Referências bibliográficas 21

Lista de Figuras

3.1 Diagrama de arquitetura básico 8
3.2 ElasticStack 9
3.3 Tela inicial do Kibana 14

4.1 Requisição como vista no Kibana 16
4.2 Listagem de serviços do APM 17
4.3 Diagrama de latências 18

Capítulo 1

Introdução

1.1 Objetivos e motivação

A evolução das aplicações de *Machine Learning* (ML) tem se destacado como um campo fértil para inovação tecnológica, permeando diversos setores com soluções que vão desde sistemas de recomendação até assistentes virtuais inteligentes. No entanto, para explorar de forma mais eficiente a implementação dessas soluções em produção, é necessário um conjunto robusto de práticas e processos que a Engenharia de Software levou décadas para desenvolver. A maturidade da aplicação desses princípios na área de ML ainda está em desenvolvimento, e a comunidade de cientistas de dados busca constantemente se alinhar às melhores práticas de engenharia para garantir a eficácia e a confiabilidade dos sistemas baseados em ML.

Um projeto de ML de ponta-a-ponta desde o momento de levar a produção, entrega, mensurar sua performance como um um todo e sua capacidade de gerar valor é uma tarefa complexa e árdua (Symeonidis et al. 2022).

Este trabalho se propõe a abordar a lacuna existente entre a Engenharia de Software e o avanço das técnicas de ML, enfatizando a importância da observabilidade e da instrumentação no contexto das aplicações de *Machine Learning*. A observabilidade, um conceito chave na gestão de sistemas complexos, permite monitorar e entender o comportamento de um modelo em produção, identificando gargalos e oportunidades de melhoria. A instrumentação, por sua vez, envolve a inclusão de mecanismos que possibilitam a coleta de dados de performance essenciais para a observabilidade.

Com a necessidade de padronizações e uma unificação surgiu o paradigma, inspirado pelo conceitos de *continuous integration* (CI) and *continuous deployment* (CD), o conceito de MLops é extendido a partir do DevOps, com a tarefa de melhorar a eficiência e robustas na entrega de produtos de ML(Bodor, Hnida & Daoudi 2023).

No ambiente de produção se faz necessario definir métricas de performance que podem não ser diretamente ligadas ao modelo mas ainda são importantes para garantir a qualidade e avaliar o modelo no ambiente. As métricas comuns incluem *query Rate*, tempo de resposta, latência, velocidade de previsão, consumo de recursos, etc (Bodor, Hnida & Daoudi 2023)

Um novo conceito que surge nesse contexto é o de Continuous Monitoring (CM) (Bodor, Hnida & Najima 2023), em que a saúde dos modelos em produção deve ser levada

em conta. Nesse espaço, esse trabalho se propõe a analisar e implementar uma abordagem para tal problema. Uma aplicação a ser desenvolvida e métricas serão coletadas, bem como a avaliação do potencial da solução na construção de um arcabouço unificado para MLops

1.2 Estrutura do trabalho

Além dessa introdução, este trabalho está dividido em mais 4 capítulos. Capítulo 2 abordara sobre as tecnologias e ferramentas utilizadas. Capítulo 3 sobre a implementação e detalhes sobre o projeto em si. Capítulo 4 irá abordar sobre como os experimentos foram realizados e os dados coletados. O Capítulo 6 abordará sobre conclusões e discussões sobre o potencial da ferramenta de acordo com os objetivos enumerados na introdução.

Capítulo 2

Fundamentação Teórica

Aqui, serão explorados os fundamentos dos contêineres Docker, que permitem a consistência e escalabilidade das aplicações; o Elasticsearch, Kibana e APM server, que juntos formam o Elastic Stack, oferecendo uma solução robusta para coleta, armazenamento e visualização de métricas de performance; e o FastAPI, um framework moderno e eficiente para a criação de APIs RESTful em Python. Esses pilares teóricos e tecnológicos são essenciais para compreender como a observabilidade pode ser implementada e como os dados coletados podem ser utilizados para otimizar e garantir a confiabilidade de serviços de inferência de Machine Learning.

2.1 Contêineres Docker

Docker é uma plataforma de virtualização ao nível de sistema operacional, baseada em contêineres, que permite o desenvolvimento, o teste e a implantação de aplicações de forma consistente em diferentes ambientes. Utiliza a tecnologia de contêineres do Linux para empacotar o software em um ambiente completo, que inclui todos os seus componentes e dependências, garantindo que a aplicação funcione da mesma maneira independentemente do ambiente em que é executada.

Para aplicações de Machine Learning, o Docker é particularmente útil, pois facilita a reprodução de ambientes de desenvolvimento em ambientes de produção, minimiza conflitos de dependência e permite a rápida escalabilidade de serviços. Além disso, o Docker é um pilar para a implementação de pipelines de CI/CD, automatizando a implantação e os testes de aplicações.

2.2 Elasticsearch, Kibana e APM server

O Elasticsearch é um sistema de busca e análise distribuído, livre e open-source, que se baseia na biblioteca de busca Lucene. Projetado para ser altamente escalável e fácil de operar, o Elasticsearch é frequentemente utilizado como a espinha dorsal de sistemas de monitoramento e logística de dados, devido à sua capacidade de indexar e buscar grandes volumes de dados com eficiência.

No contexto de observabilidade e gestão de performance de aplicações, como é o caso da inference-api, o Elasticsearch desempenha um papel fundamental ao servir como repositório central para logs e dados de APM (Application Performance Management). A ferramenta de APM, ElasticAPM, coleta informações detalhadas sobre o desempenho da aplicação, incluindo métricas de transações, erros e estatísticas de utilização de recursos, e armazena esses dados no Elasticsearch.

A importância do Elasticsearch nesse cenário reside em sua habilidade de lidar com a ingestão e o armazenamento de uma grande quantidade de dados gerados continuamente pela instrumentação da API. Esses dados incluem *logs* de transações, métricas de performance e qualquer incidente de erro que ocorra durante a operação de APIs REST. A capacidade de armazenar e indexar esses dados de forma estruturada permite que sejam realizadas buscas complexas e análises em tempo real, o que é essencial para a detecção de anomalias de performance e para a tomada de decisão baseada em dados concretos.(Kathare et al. 2020)

Além disso, o Elasticsearch é parte da Elastic Stack, que inclui o Kibana, uma interface de usuário que permite a visualização e análise interativa dos dados armazenados no Elasticsearch (Akdal et al. 2018). Com o Kibana, é possível criar dashboards personalizados que exibem métricas relevantes para o monitoramento da APIs REST, facilitando a identificação de padrões e tendências que podem indicar a necessidade de otimizações.

De acordo com o artigo "A Basic Introduction to DevOps Tools", o autor conclui que um *stack* formado por Elasticsearch e Kibana pode ser uma boa alternativa (Jagadish 2015), tornando essa a ferramenta mais versátil e o melhor candidato para os objetivos na questão de monitoramento.

Outros casos de de uso do ElasticStack para monitoramento é descrito em "Monitoring of IaaS and scientific applications on the Cloud using the Elasticsearch ecosystem"(Bagnasco et al. 2015), no qual dados e uma Cloud privada é ingerido no ElasticSearch

Alem disso, outro caso de sucesso utilizou o cliente ElasticAPM, para melhorar a observabilidade em sistemas SCADA complexos, em que a quantidade de dados crescia exponencialmente. No paper, os autores concluem que a abordagem de integrar a ferramenta ofereceu modularidade e *insights* imediatos. (Leitner et al. 2012)

Em resumo, o Elasticsearch é uma peça central na infraestrutura de monitoramento de aplicações de Machine Learning, oferecendo uma solução robusta para armazenamento, busca e análise de logs e dados de performance. Sua integração com ferramentas de APM como o ElasticAPM permite um monitoramento abrangente e aprofundado, essencial para manter a saúde e a performance de sistemas críticos.

2.3 FastAPI

A arquitetura REST (Representational State Transfer) é um estilo de design para sistemas de software distribuídos, que se tornou padrão para o desenvolvimento de APIs web devido à sua simplicidade e capacidade de promover a interação *stateless* entre clientes e servidores. APIs REST são projetadas para serem intuitivas, baseando-se em operações de HTTP que correspondem às ações naturais de criação, leitura, atualização e exclusão

de recursos (CRUD).(*Fielding Dissertation: CHAPTER 5: Representational State Transfer (REST) — ics.uci.edu* n.d.)

No contexto de uma "API de inferência"genérica para Machine Learning, a utilização de um framework RESTful como o FastAPI oferece vantagens significativas. FastAPI é conhecido por sua eficiência e por sua capacidade de tornar o desenvolvimento de APIs com Python mais ágil e menos propenso a erros, graças à sua integração com sistemas de tipagem do Python e ferramentas de validação de dados. Além disso, ela pode ser até 45% mais performática em modelos de ML do que se usado frameworks como Flask(Bansal & Ouda 2022), o que faz o framework ideal para a construção de uma API REST.

Um serviço de inferência desenvolvida com FastAPI pode expor endpoints que permitem a interação com modelos de ML, aceitando dados de entrada e retornando predições ou análises. A auto-documentação gerada pelo FastAPI, através de interfaces como o Swagger UI, facilita o entendimento e a utilização da API por terceiros, promovendo a interoperabilidade e a reutilização de código.

A natureza *stateless* das APIs REST, como a que seria implementada para a "API de inferência", é ideal para ambientes de nuvem e escalabilidade horizontal, onde múltiplas instâncias de um serviço podem ser distribuídas para atender a demandas variáveis de solicitações. Cada instância pode processar requisições independentemente, sem a necessidade de compartilhar estado entre si, o que simplifica o design e a manutenção do sistema.

Quando integrada com ferramentas de APM como o ElasticAPM, uma "API de inferência"baseada em FastAPI pode ser monitorada em tempo real, permitindo a coleta de métricas críticas, como o tempo de resposta de cada endpoint, a utilização de recursos do servidor e a identificação de possíveis pontos de falha ou lentidão. Esses dados são então armazenados e indexados pelo Elasticsearch, possibilitando consultas e análises detalhadas que auxiliam na otimização da performance da API.

Em resumo, a utilização de uma API RESTful com FastAPI para o desenvolvimento de um serviço de inferência baseado de Machine Learning proporciona uma solução robusta e escalável, ao mesmo tempo em que facilita a implementação de práticas de monitoramento e observabilidade essenciais para a Engenharia de Software e DevOps. Isso assegura que a "API de inferência"não apenas ofereça uma interface acessível e bem documentada para os modelos de ML, mas também mantenha um alto nível de performance e confiabilidade em produção.

Capítulo 3

Desenvolvimento

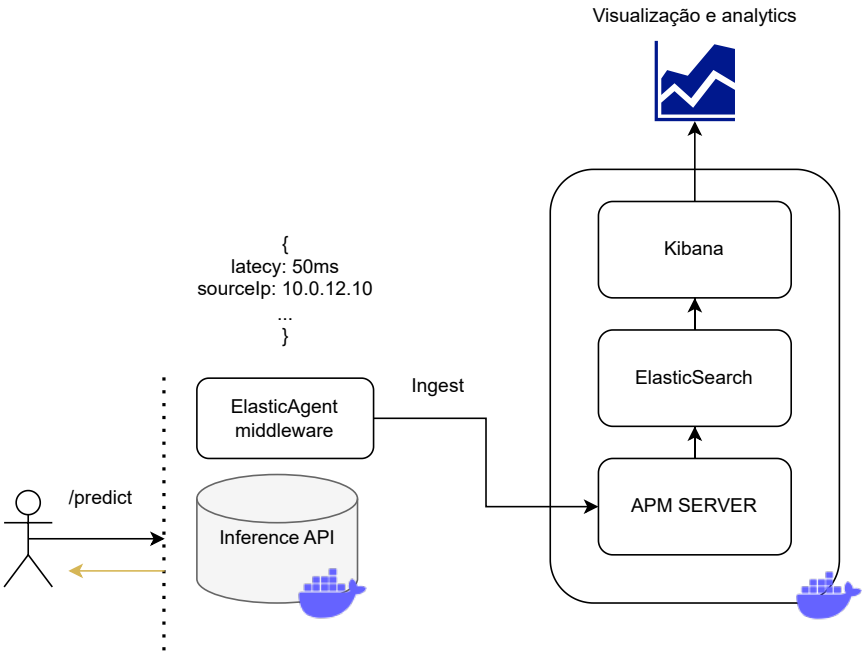
3.1 Desenvolvimento da API REST

A API REST foi desenvolvida utilizando a linguagem Python e o framework FastAPI, reconhecido por sua facilidade de uso e capacidades assíncronas, que são ideais para servir endpoints que realizam operações de machine learning. O projeto centraliza-se no endpoint */predict*, que é responsável por aceitar um conjunto de valores de entrada e passá-los por um modelo de machine learning de exemplo, fornecido pela biblioteca scikit-learn. Este modelo é um exemplo simplificado e serve apenas para demonstrar a funcionalidade da API, não sendo o foco principal do trabalho.

```python
from fastapi import FastAPI
from pydantic import BaseModel
from typing import List
from sklearn.datasets import load_iris
from sklearn.linear_model import LinearRegression
import numpy as np

app = FastAPI()

class PredictionInput(BaseModel):
    features: List[float]

iris = load_iris()
model = LinearRegression()
model.fit(iris.data, iris.target)

@app.post("/predict")
async def predict(input_data: PredictionInput):

    prediction = model.predict([input_data.features])
    return {"prediction": prediction.tolist()}

@app.get("/")
async def root():
    return {"message": "API is operational"}
```

Código 3.1: Código API implementado em *main.py*

Figura 3.1: Diagrama de arquitetura básico



Fonte: Elaborado pelo Autor

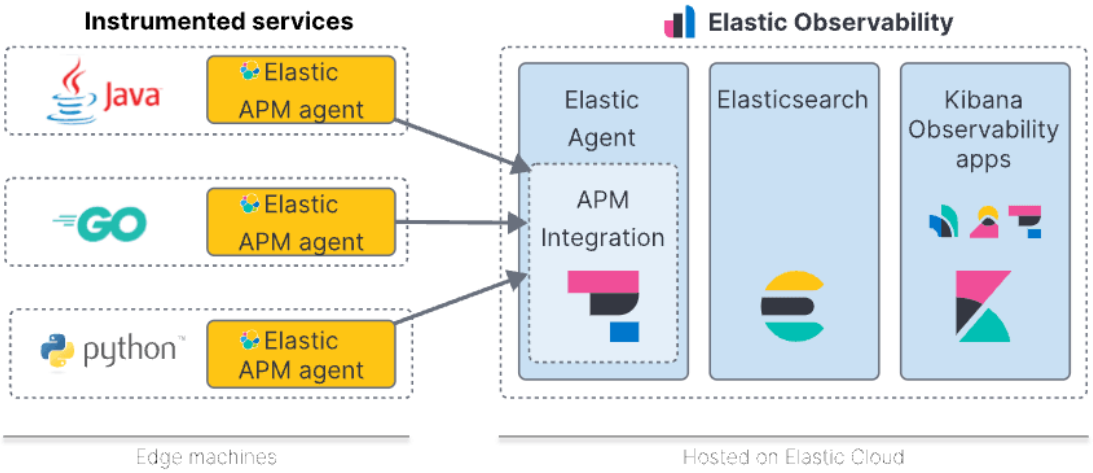
No serviço de inferência, o agente, dentro do contêiner, intercepta cada requisição, mede as latências nos trechos de código mostrado em 3.1, a informação é enviada em um documento Json para o *APM SERVER*, que posteriormente é indexado nos índices do ElasticSearch, um vez lá é possível utilizar o Kibana para monitorar em tempo real o serviço de inferência.

A utilização do FastAPI também facilitou a tipagem de dados e a validação automática dos inputs recebidos pelo endpoint */predict*, o que é essencial para garantir que o modelo de machine learning receba os dados corretos e em formato adequado para realizar suas predições. Além disso, a documentação automática gerada pelo FastAPI, através de interfaces como Swagger UI, permite que os usuários da API entendam claramente como enviar as requisições e o que esperar das respostas, melhorando a usabilidade e a manutenção do serviço.

O código 3.1 exemplifica como um modelo de machine learning pré-treinado do *Scikit-learn* pode ser integrado a uma API REST usando FastAPI. No caso, um modelo *LinearRegression* treinado com o conjunto de dados Iris (Fisher 1936) (conjunto de dados que vem por padrão na biblioteca) é utilizado para realizar predições. O endpoint */predict* espera que os dados de entrada sejam fornecidos como uma lista de características (features), que correspondem às variáveis de entrada do modelo de ML. Após receber os dados, o modelo faz a predição, e o resultado é retornado em formato JSON.

A API foi desenhada para ser eficiente e fácil de manter, com a validação de dados de entrada garantindo que apenas informações adequadas sejam processadas pelo modelo. A

Figura 3.2: ElasticStack



Fonte: elastic.co

estrutura assíncrona do FastAPI está pronta para lidar com operações intensivas em I/O, como seria o caso com modelos de ML mais complexos e conjuntos de dados maiores.

Embora o modelo de regressão linear e o conjunto de dados Iris sejam simples e não representem a complexidade de muitos problemas reais de machine learning, eles são suficientes para demonstrar a funcionalidade e a arquitetura da API. O monitoramento deste *endpoint* com o ElasticAPM permitirá a coleta de métricas valiosas sobre o desempenho da API, como tempos de resposta e taxa de sucesso das requisições, essenciais para a análise de observabilidade e para a tomada de decisões em futuras otimizações e escalabilidade do serviço.

3.2 Integração com ElasticAPM

A implementação do monitoramento de performance da API REST de *machine learning* com ElasticAPM envolveu a configuração e integração de uma série de componentes dentro de um ambiente Dockerizado. Foi utilizada as imagens oficiais disponibilizadas pela Elasticsearch, que já incluem a orquestração necessária para levantar contêineres do Elasticsearch e do Kibana, este último com o agente elastic. Este setup permite uma rápida prototipagem e a obtenção de *insights* sobre o comportamento da API.

O elastic-agent é uma integração do Kibana, ele tem a função de coletar as informações do código instrumentalizado do container e armazená-las no ElastiSearch, desse modo, os dados ficarão acessíveis utilizando o Kibana.

Como pode ser observado, é definida uma senha básica para o ElasticSearch utilizando a variável de ambiente ELASTIC_PASSWORD. Além disso, um contêiner efêmero chamado *setup* foi utilizado com o propósito de criar configurações básicas para o funcionamento do ElasticSearch, tais como criação de roles extras, credenciais, índices extras ou política de retenção. Para o propósito deste trabalho as configurações padrões e mais básicas foi utilizada.[1]

Para configurar o Kibana e o apm-agent, foram criados dois arquivos e , em seguida, mapeado para dentro dos respectivos contêineres. Os arquivos *kibana.yml* e *apm-server.yml* nos permitem um grande controle e variedade de configurações, para o uso básico, foi necessário apenas configurar *host* e credenciais para os serviços descobrirem e se comunicarem com o serviço ElasticSearch.

[1] O ElasticStack conteinerizado está disponível no repositório: https://github.com/elastic/apm-server/blob/main/docker-compose.yml e o *docker-compose.yml* está disponível na integra nos anexos

```

        apm-server:

            host: "0.0.0.0:8200"
            secret_token: CHAVE_SECRETA_COMPARTILHADA

        elasticsearch:
            `http://elasticsearch:9200/path`.

            hosts: ["elasticsearch:9200"]

            # Authentication credentials - either API key or username/
    password.
            #api_key: "id:api_key"
            username: "elastic"
            password: "SECURE_PASSWORD"
```

Código 3.2: Arquivo *apm-server.yml*

Na configuração, foi optada pela utilização de um token secreto simples, no entanto é possível habilitar o uso de chaves de API do próprio ElasticSearch, a chave CHAVE_SECRETA_COMPARTILHADA, irá ser utilizada nos agentes para instrumentação de código.

Em seguida, é utilizado o utilizado um arquivo de configuração para habilitar o APM na *User Interface* do Kibana. Além de permitir com que o proprio Kibana possa se comunicar com o ElasticSearch, através de usuários específicos, se definidos. Para simplicidade, foi utilizado o usuário padrão do ElasticSearch criado na inicialização do mesmo.

```
server.name: kibana
server.host: 0.0.0.0
elasticsearch.hosts: [ http://elasticsearch:9200 ]

monitoring.ui.container.elasticsearch.enabled: true
monitoring.ui.container.logstash.enabled: true

## X-Pack security credentials
#
elasticsearch.username: kibana_system
elasticsearch.password: ${KIBANA_SYSTEM_PASSWORD}

## Fleet
## https://www.elastic.co/guide/en/kibana/current/fleet-settings-kb.
    html
#
xpack.fleet.agents.fleet_server.hosts: [ http://fleet-server:8220 ]

xpack.fleet.outputs:
  - id: fleet-default-output
    name: default
    type: elasticsearch
    hosts: [ http://elasticsearch:9200 ]
    is_default: true
    is_default_monitoring: true
```

```yaml


- name: Agent Policy APM Server
  id: agent-policy-apm-server
  description: Static agent policy for the APM Server integration
  monitoring_enabled:
    - logs
    - metrics
  package_policies:
    - name: system-1
      package:
        name: system
    - name: elastic_agent-1
      package:
        name: elastic_agent
    - name: apm-1
      package:
        name: apm

      inputs:
        - type: apm
          vars:
            - name: host
              value: 0.0.0.0:8200
            - name: url
              value: http://apm-server:8200
```

Código 3.3: Arquivo *kibana.yml*

Uma vez que todos os serviços estejam configurados, os serviços foram inicializados, e, então foi possível acessar o Kibana através do navegado pelo endereço da porta mapeado no arquivo *docker-compose.yml*

3.3 Dockerização dos Serviços e instrumentação de código

Uma vez que os serviços de monitoramento e observabilidade foram criados com êxito, pequenas mudanças no código 3.1 para este possa se comunicar com o servidor APM. Para integrar o ElasticAPM no serviço de ML, foi necessário adicionar o agente APM para Python no código da aplicação, adicionando o pacote elastic-apm na versão 6.7.1 nos arquivos *requirements.txt*. O agente é configurado para se conectar ao APM SERVER através de variaveis de ambiente, permitindo o rastreamento automático das requisições HTTP e a geração de spans para cada operação realizada pelo endpoint */predict*.

```python
from fastapi import FastAPI
from elasticapm.contrib.starlette import make_apm_client, ElasticAPM
from pydantic import BaseModel
from typing import List
from sklearn.linear_model import LinearRegression
import joblib
import numpy as np

    # Configuracao do ElasticAPM
apm_client = make_apm_client({
    'SERVICE_NAME': 'ml_api',
    'ENVIRONMENT': 'prod',
    'SERVER_URL':os.getenv('APM_SERVER_HOST'),
    'SECRET_TOKEN': os.getenv('APM_SECRET_TOKEN')
    })

app = FastAPI()
app.add_middleware(ElasticAPM, client=apm_client)


class PredictionInput(BaseModel):
    features: List[float]

model = joblib.load("linear_regression_model.joblib")

@app.post("/predict")
async def predict(input_data: PredictionInput):
    with apm_client.capture_span("process_prediction")
        prediction = model.predict([input_data.features])
    return {"prediction": prediction.tolist()}

@app.get("/")
async def root():
    return {"message": "API is operational"}
```

Código 3.4: Arquivo *app.py* modificado

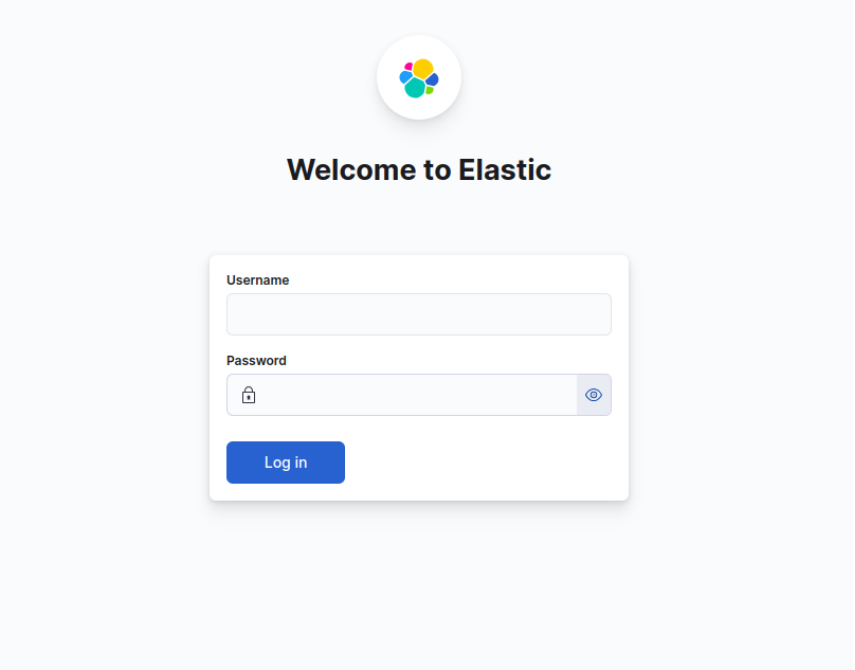
O modelo de ML foi carregado como artefato no próprio contêiner da aplicação, ele será carregado diretamente no inicio na inicialização da API para ser mais performático, reduzindo o tempo de inicialização.

Na nova versão, é criado um *middleware*, isso é onde toda as requisições serão direcionadas, o pacote do *elasticapm* irá capturar informações de *span*, tais informações são enviadas para o servidor APM, este que por sua vez, retêm em índices no ElasticSerach. Isso permite um rastreamento detalhado da operação de predição, desde o momento em que o *span* é iniciado até o seu término, oferecendo *insights* sobre o tempo de processamento dedicado à inferência do modelo de machine learning.

A abordagem adotada neste trabalho destaca a importância da observabilidade em aplicações de machine learning servidas via *REST*, demonstrando como o ElasticAPM pode ser efetivamente utilizado para capturar e analisar spans críticos, mesmo em um exemplo didático com um modelo *LinearRegression* do pacote scikit-learn.

Com o proposito de integração, podemos modificar o arquivo docker-compose em

Figura 3.3: Tela inicial do Kibana



Fonte: Elaborado pelo Autor

?? para que este possa descobrir o contêiner da servidor APM, tendo em vista que o os contêineres se encontraram em uma mesma rede. É importante observar que a o token secreto pre-compartilhado deve ser o mesmo configurado no servidor APM.

```
      fastapi_app:
    build: ./api
    volumes:
      - ./api:/app  # Montar o volume para incluir o codigo da API e
   requirements.txt
    environment:

      - APM_SERVER_HOST=http://apm-server:8200
      - SERVICE_NAME=ml_api
      - SECRET_TOKEN=TOKEN_SECRETO
    depends_on:
      - apm-server
    ports:
      - "8000:8000"
```

Código 3.5: Modificação feita no *docker-compose.yml*

Capítulo 4

Experimentos e Resultados

A fase de experimentos e resultados é crucial para validar a eficácia da implementação da observabilidade na API REST de *machine learning* utilizando o ElasticAPM em um ambiente Dockerizado. Nesta seção, é detalhado os procedimentos experimentais realizados, a análise dos dados coletados, as conclusões preliminares sobre o desempenho e a escalabilidade da API e as visualizações resultantes no Kibana.

4.1 Configuração Experimental

A configuração do ambiente experimental envolveu a preparação de um conjunto de dados de entrada que refletissem o uso esperado do endpoint */predict*. O modelo de regressão linear pré-treinado, carregado na aplicação, foi submetido a uma série de requisições contendo diferentes conjuntos de características, representativos de cenários reais, realizando em torno de 200 requisições para a API, em seguida, verificou-se o comportamento na interface do Kibana. O proposito era averiguar se as informações trazidas por ele podem ser relevantes ao

As solicitações HTTP do tipo POST para o endpoint */predict* foram elaboradas para refletir o uso típico esperado, com o corpo da requisição em formato JSON.

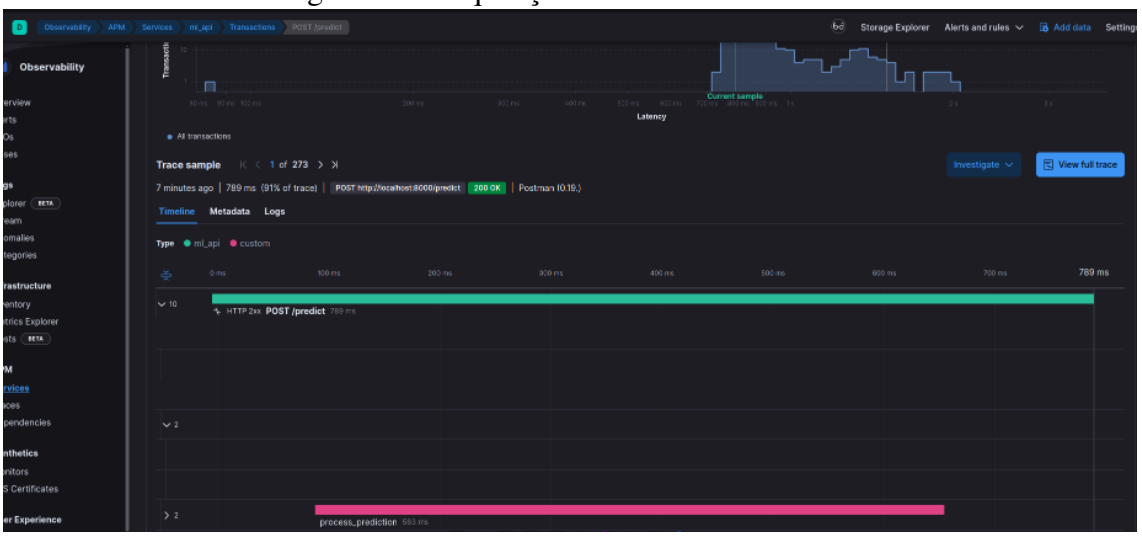
```
{
    "features": [5.1, 3.5, 1.4, 0.2]
}
```

Dados Serializado 1: formato de JSON esperado pela API

4.2 Visualização no Kibana

No Kibana, criamos visualizações para exibir o tempo de resposta médio, a distribuição dos tempos de processamento do span *process_prediction*, e a frequência das requisições ao longo do tempo. Essas visualizações ajudaram a identificar rapidamente qualquer anomalia no comportamento da API, como latências inesperadas ou erros de processamento.

Figura 4.1: Requisição como vista no Kibana



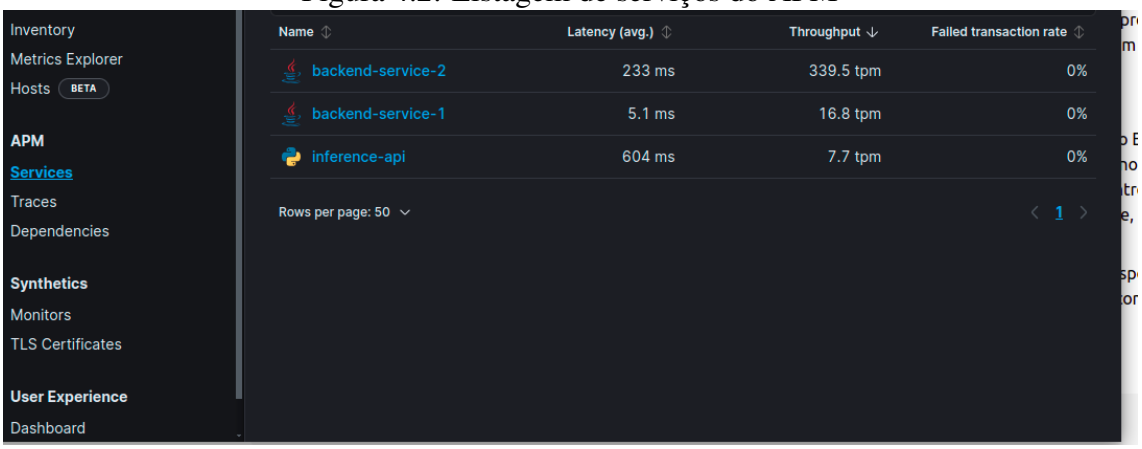
Fonte: Elaborado pelo autor

A figura 4.1 ilustra a latência total de toda a requisição (barra verde), bem como a latência de um trecho de código especifico (barra vermelha), desse modo é possível identificar facilmente onde os principais gargalos se encontram.

Além disso, o tipo de requisição (POST), o *endpoint* atingido (*/predict*) e o cliente utilizado (Postman) são capturados e aparecem na interface, permitindo fazer analises de correlação entre as variáveis.

Uma principais vantagens do uso do ElasticAPM em conjunto com o Kibana é a capacidade de oferecer uma visualização unificada de múltiplos serviços dentro de uma arquitetura de microserviços. No contexto dos experimentos realizados, além da nossa aplicação FastAPI denominada *inference-api*, também é possível monitorar e visualizar dois serviços adicionais em Java, demonstrando a flexibilidade e a amplitude da solução de monitoramento. Os serviços que aparecem na imagem são apenas ilustrações da capacidade da aplicação, não sendo serviços realmente funcionais ou relevantes no trabalho.

Figura 4.2: Listagem de serviços do APM



Fonte: Elaborado pelo auto

4.3 *Insights* para Otimização

A visualização no Kibana, a partir dos dados coletados pelo ElasticAPM, destacou informações cruciais que poderão ser utilizadas para futuras otimizações na inference-api. A análise focou na distribuição de tempos de resposta e no comportamento do endpoint */predict* sob condições normais de operação.

Os *spans* detalhados gerados pelo ElasticAPM permitiram a observação precisa das etapas envolvidas no processamento de cada requisição POST na inference-api. Isso incluiu o tempo gasto no carregamento do modelo de *machine learning*, a execução da predição e qualquer interação com outros componentes da aplicação, como bancos de dados ou serviços de armazenamento.

Esses dados de monitoramento servem como uma base sólida para identificar oportunidades de melhoria. Por exemplo, caso se observe um padrão de latência elevada na etapa de predição, isso poderia indicar a necessidade de investigar a eficiência do modelo de *machine learning*. Alternativas como a otimização do código do modelo, a utilização de hardware mais potente, ou a exploração de técnicas de paralelização poderiam ser consideradas.

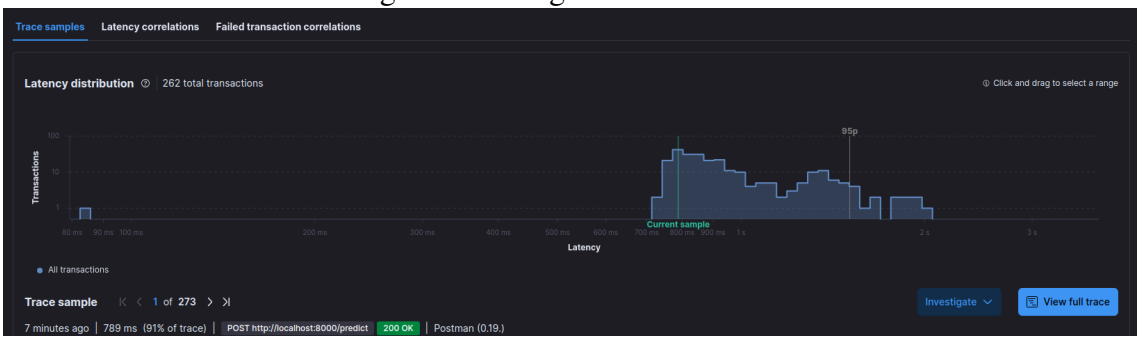
Da mesma forma, se os dados mostrarem tempos de resposta inconsistentes ou longos em operações de entrada/saída, como leitura de dados de arquivos ou comunicação com o Elasticsearch, isso sugeriria uma revisão das estratégias de gerenciamento de recursos e conexões. A implementação de pools de conexão mais eficientes ou a otimização das consultas poderiam ser ações subsequentes para melhorar a performance.

Ainda que os serviços Java tenham sido mencionados apenas para ilustrar a capacidade do ElasticAPM e do Kibana de integrar múltiplos serviços, a atenção foi exclusivamente na *inference-api*. A visualização dos dados dessa aplicação específica no Kibana é suficiente para revelar pontos de interesse que, uma vez analisados, podem conduzir a otimizações significativas na arquitetura e no código do serviço de inferência.

Esses *insights* preliminares sublinham o valor do ElasticAPM como uma ferramenta estratégica de monitoramento, pronta para fornecer os dados necessários para que, em

etapas subsequentes, as otimizações na *inference-api* sejam conduzidas de maneira informada, visando a melhoria contínua da performance e a eficácia operacional do sistema.

Figura 4.3: Diagrama de latências



Fonte: Elaborada pelo autor

A Figura 4.3 apresenta um diagrama que ilustra a distribuição das latências das transações, onde o eixo y representa a quantidade de requisições que levaram um determinado tempo para serem processadas. A concentração de transações no lado esquerdo do gráfico é o cenário ideal, pois evidencia um alto *throughput*, ou seja, um grande volume de requisições sendo atendidas rapidamente. No sistema real, buscamos por um limiar adequado de acordo com critérios definidos, por exemplo respostas abaixo de 1.5s.

Capítulo 5

Conclusão

O trabalho desenvolvido teve como um de seus principais objetivos aproximar o cientista de dados das práticas modernas de desenvolvimento de software, com ênfase especial em instrumentação, observabilidade e arquitetura de microserviços. A aplicação dessas práticas na inference-api, uma API REST desenvolvida com FastAPI, demonstrou como tais abordagens podem ser benéficas para o monitoramento e a análise de performance em sistemas de machine learning.

A instrumentação da *inference-api* com o ElasticAPM permitiu a coleta detalhada de métricas de performance, oferecendo aos cientistas de dados uma ferramenta poderosa para entender o comportamento de um serviço de inferência em tempo real. A visualização desses dados no Kibana proporcionou *insights* valiosos, que podem ser utilizados para guiar otimizações futuras, como a melhoria da eficiência do modelo de *machine learning* ou a otimização das operações de entrada/saída.

A adoção de práticas de observabilidade destacou a importância de se ter uma visão clara e detalhada do funcionamento interno das aplicações, algo frequentemente negligenciado em projetos de ciência de dados. Isso abre um leque de oportunidades para que cientistas de dados se envolvam mais profundamente com aspectos de engenharia de software, resultando em soluções mais robustas e eficientes.

A implementação da inference-api como um microserviço em contêiner exemplificou como as APIs de machine learning podem ser desacopladas, escaláveis e facilmente gerenciáveis. Essa abordagem facilita a integração contínua e a entrega contínua (CI/CD), promovendo um ciclo de desenvolvimento mais ágil e alinhado às necessidades de mudança e adaptação rápidas que o ambiente de dados muitas vezes exige.

Embora o escopo deste trabalho não tenha incluído a realização de testes de carga extensivos, ficou evidente a necessidade de se aplicar tais testes para avaliar a escalabilidade da *inference-api* sob demanda elevada. A infraestrutura de monitoramento estabelecida está pronta para suportar essa fase de testes, fornecendo uma base de dados confiável para análise.

Em conclusão, este trabalho cumpriu seu objetivo de integrar práticas de desenvolvimento de software ao domínio da ciência de dados, demonstrando o valor da observabilidade e da arquitetura de microserviços. Recomenda-se que, em trabalhos futuros, essas práticas sejam expandidas e aprofundadas, incluindo a realização de otimizações baseadas nos dados de monitoramento e a execução de testes de carga para avaliar a performance sob diferentes cenários. A documentação e disseminação dessas práticas entre

a comunidade de cientistas de dados podem contribuir significativamente para a evolução da implementação e operacionalização de modelos de *machine learning*.

Referências Bibliográficas

Akdal, Berkay, Zehra Gül Çabuk Keskin, Erdem Eser Ekinci & Geylani Kardast (2018), Model-driven query generation for elasticsearch, *em* '2018 Federated Conference on Computer Science and Information Systems (FedCSIS)', pp. 853–862.

Bagnasco, S, D Berzano, A Guarise, S Lusso, M Masera & S Vallero (2015), 'Monitoring of iaas and scientific applications on the cloud using the elasticsearch ecosystem', *Journal of Physics: Conference Series* **608**(1), 012016.
URL: *https://dx.doi.org/10.1088/1742-6596/608/1/012016*

Bansal, Priya & Abdelkader Ouda (2022), Study on integration of fastapi and machine learning for continuous authentication of behavioral biometrics, *em* '2022 International Symposium on Networks, Computers and Communications (ISNCC)', pp. 1–6.

Bodor, Anas, Meriem Hnida & Daoudi Najima (2023), Mlops: Overview of current state and future directions, *em* M.Ben Ahmed, A. A.Boudhir, D.Santos, R.Dionisio & N.Benaya, eds., 'Innovations in Smart Cities Applications Volume 6', Springer International Publishing, Cham, pp. 156–165.

Bodor, Anas, Meriem Hnida & Najima Daoudi (2023), 'Machine learning models monitoring in mlops context: Metrics and tools', *International Journal of Interactive Mobile Technologies (iJIM)* **17**(23), pp. 125–139.
URL: *https://online-journals.org/index.php/i-jim/article/view/43479*

Fielding Dissertation: CHAPTER 5: Representational State Transfer (REST) — ics.uci.edu (n.d.), https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm. [Accessed 16-01-2025].

Fisher, R. A. (1936), 'Iris', UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C56C76.

Jagadish, N. (2015), A basic introduction to devops tools.
URL: *https://api.semanticscholar.org/CorpusID:204780604*

Kathare, Nikita, O Vinati Reddy & Vishalakshi Prabhu (2020), 'A comprehensive study of elasticsearch', *International Journal of Science and Research (IJSR)* .

Leitner, Philipp, Christian Inzinger, Waldemar Hummer, Benjamin Satzger & Schahram Dustdar (2012), Application-level performance monitoring of cloud services based on the complex event processing paradigm, *em* '2012 Fifth IEEE International Conference on Service-Oriented Computing and Applications (SOCA)', pp. 1–8.

Symeonidis, Georgios, Evangelos Nerantzis, Apostolos Kazakis & George A. Papakostas (2022), Mlops - definitions, tools and challenges, *em* '2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC)', pp. 0453–0460.